

Software Process Models

1. What Are Prescriptive Process Models?

Prescriptive process models are the traditional software process models that define a structured, disciplined approach to software development. They *prescribe* a set of framework activities, tasks, work products, quality-assurance steps, and change-control mechanisms, along with a predictable workflow — hence the name.

These models were introduced to bring order and consistency to software development, which in its early days was chaotic and unorganized. They emphasize detailed planning, documentation, and control to produce predictable results.

However, real software development often exists "at the edge of chaos" — too much rigidity reduces flexibility and innovation, while too little structure leads to disorder. Prescriptive models work well when requirements are well-defined and stable, but can become inefficient (or outright counterproductive) in rapidly changing environments.

Why can prescriptive models become bureaucratic? When applied too rigidly, the documentation, approvals, and formal procedures that were meant to *support* development can end up slowing it down instead — reducing creativity and resisting legitimate change. Instead of the process serving development, development starts serving the process, which leads to inefficiency and developer frustration. This is the central critique that later motivated agile approaches.

2. The Waterfall Model

The **Waterfall Model** is the oldest and most widely known prescriptive process model. It follows a strictly **linear and sequential** approach — each phase must be fully completed before the next one begins.

Communication → Planning → Modeling → Construction → Deployment
(requirements) (analysis & design) (coding & testing) (delivery & maintenance)

Phases

1. **Communication** — requirements gathering
2. **Planning** — project planning
3. **Modeling** — analysis and design

4. **Construction** — coding and testing
5. **Deployment** — delivery and maintenance

Advantages

- Simple and easy to understand
- Clear documentation and milestones at every phase boundary
- Suitable for small projects with stable, well-understood requirements

Disadvantages

- Difficult to accommodate changing requirements — going "backward" a phase is expensive and often resisted
 - Working software is delivered very late (only at the very end)
 - Errors discovered late (e.g., during testing) are costly to fix, since they may trace back to a design or requirements mistake
 - Real projects rarely follow a strictly linear sequence — some backtracking is almost always needed
-

3. The Prototyping Process Model

The Prototyping Model is used when requirements are **not clearly defined or are likely to change**. Instead of building the complete system in one shot, a working prototype is built quickly to help users visualize and understand the system. Users evaluate the prototype and give feedback, which is used to refine requirements through multiple iterations — helping both developers and users converge on a clearer picture of what the final system should actually be.

Advantages

- Helps clarify unclear or incomplete requirements
- Users can see and interact with the system early
- Reduces misunderstandings between users and developers
- Improves user satisfaction through continuous feedback

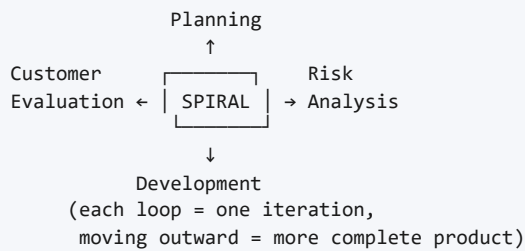
Disadvantages

- Users may mistake the prototype for the finished, production-ready system
 - Developers may take quick design shortcuts (to build the prototype fast) that end up compromising quality if reused as-is
 - Documentation is often poor, since the focus is on speed
 - Not well suited to large, highly complex systems
-

4. The Evolutionary Process Model (Spiral Model)

The Evolutionary Process Model develops software as a series of **incremental and iterative releases** — the system evolves over time as new features are added based on customer feedback.

The **Spiral Model** is the best-known evolutionary model. It combines iterative development with systematic planning and risk analysis. Each loop ("cycle") around the spiral includes four activities: **planning, risk assessment, development (engineering), and customer evaluation** — and the product improves gradually with each loop.



Advantages

- Accommodates changing requirements easily
- Delivers working software early, then incrementally improves it
- Continuous customer feedback improves quality release over release
- Explicit risk analysis in every cycle helps prevent major project failures

Disadvantages

- Requires experienced developers, specifically for the risk-management portion of each cycle
- Can be costly and time-consuming compared to a straightforward linear model
- Difficult to manage for small projects — the overhead of a full spiral cycle isn't justified
- Customers may find the open-ended, cyclical process harder to understand than a fixed linear plan with a visible end date

5. The Unified Process (UP) Model

The **Unified Process** is an iterative and incremental process model that combines the strengths of traditional prescriptive models with agile principles. It emphasizes customer communication, **use-case-driven development**, and a strong software architecture. UP uses UML for modeling and supports evolutionary development through repeated software increments.

The Five Phases of UP

Phase	Focus	What Happens
1. Inception	Communication + planning	Identify basic business requirements using preliminary use cases, assess major risks, produce a rough project plan and schedule
2. Elaboration	Requirements + architecture	Requirements are refined and expanded; a solid architectural baseline is developed using multiple views (use-case, analysis, design, implementation, deployment); plans and risks are refined further
3. Construction	Coding + testing	Software components are implemented; unit and integration testing are performed; use cases are used to derive acceptance tests for each increment
4. Transition	Beta delivery	Software is delivered to users for beta testing; feedback is collected, defects corrected, final adjustments made before release
5. Production	Operation	Software is deployed and maintained; performance is monitored, user support is provided, and change requests / defect reports are managed

Interview soundbite: "UP is essentially the same five ideas as the generic process framework — communication, planning, modeling, construction, deployment — renamed and made explicitly iterative, with 'Inception' and 'Elaboration' front-loading enough architecture and risk analysis that 'Construction' doesn't blow up later, which is exactly the failure mode Waterfall has no defense against."

Quick Comparison

Model	Best suited when...	Key risk
Waterfall	Requirements are stable, well-understood, project is small	Any late-discovered change is expensive
Prototyping	Requirements are unclear or evolving	Prototype gets shipped as the "real" product
Spiral	Large, high-risk projects needing continuous risk management	Needs experienced staff; overhead not worth it for small projects
Unified Process	Medium-to-large projects wanting architecture-first, use-case-driven iteration	Still requires discipline in Inception/Elaboration or Construction repeats Waterfall's mistakes

Interview Questions With Answers

Q1. What is a prescriptive process model, and why might it become inefficient in a rapidly changing environment?

Answer: A prescriptive process model is a traditional, structured approach to software development that lays out a specific set of framework activities, work products, quality-assurance steps, and change-control procedures in a predictable workflow, with the goal of bringing order and predictability to what would otherwise be chaotic development. It becomes inefficient in rapidly changing environments because its strength — detailed up-front planning and rigid, well-documented procedures — becomes a liability when requirements change frequently: every change has to work its way back through the formal planning and documentation machinery, which is slow, whereas the environment demands fast adaptation. The model was designed on the assumption that stability is achievable and desirable; when that assumption breaks down, the very rigor that made the model useful starts actively working against the project.

Q2. Walk through the five phases of the Waterfall Model and explain why "going backward" a phase is so costly.

Answer: The five phases are Communication (gathering requirements), Planning (creating the project plan), Modeling (analysis and design), Construction (coding and testing), and Deployment (delivery and maintenance) — and the model requires each phase to be essentially complete before the next one starts. Going backward is costly because every phase after the one being revisited was built on the assumption that the earlier phase's output was final: if a requirement gathered in Communication turns out to be wrong once you're deep into Construction, the design created in Modeling may need to change, which likely invalidates code already written in Construction, and any documentation created

along the way needs to be redone too. The cost compounds with distance — an error caught during Communication costs almost nothing to fix, but the same conceptual error caught during Deployment can require reworking design, code, and tests all at once.

Q3. When would you choose the Prototyping Model over Waterfall, and what's the main risk unique to Prototyping?

Answer: Prototyping is the better choice when requirements are unclear, incomplete, or likely to change — situations where Waterfall's assumption of stable, well-understood requirements simply doesn't hold, and building the "complete" system up front risks building the wrong thing entirely. The main risk unique to Prototyping is that users may start treating the prototype as if it were the finished, production-quality system, when in reality it was deliberately built fast, with shortcuts, specifically to validate ideas rather than to be shipped — if that quick-and-dirty version ends up becoming the actual delivered product (because deadlines slip, or because it "looks done"), all the corners that were cut to make it fast to build (poor error handling, no real documentation, weak internal design) become permanent liabilities in a production system.

Q4. Explain how the Spiral Model differs from Waterfall in how it handles risk.

Answer: Waterfall has no explicit, built-in mechanism for identifying or managing risk at all — it simply assumes the plan made at the start will hold, and if a major risk (a key technical assumption turning out to be wrong, for instance) materializes late in the project, there's no structured process for catching it early. The Spiral Model, in contrast, makes risk analysis one of the four core activities performed in *every single cycle* around the spiral, alongside planning, development, and customer evaluation — meaning the team is deliberately looking for what could go wrong at every iteration, not just once at the start. This is precisely why Spiral is recommended for large, high-risk projects: the repeated, cycle-by-cycle risk assessment can catch a fatal flaw in cycle 2 of 10, while Waterfall might not surface the same flaw until the entire system is built and testing begins.

Q5. What does "use-case-driven" mean in the context of the Unified Process, and how does UP's Inception phase try to avoid Waterfall's late-discovery problem?

Answer: "Use-case-driven" means that use cases — descriptions of how actors interact with the system to achieve specific goals — are the primary artifact used to drive requirements, design, implementation, and testing throughout the project, rather than a monolithic requirements document. Every later activity (writing acceptance tests, deriving design classes, planning increments) traces back to specific use cases. UP's Inception phase tries to avoid Waterfall's late-discovery problem by requiring that basic business requirements (via preliminary use cases) and major risks are identified *before* serious architectural work begins in Elaboration — so instead of discovering a fundamental risk or a misunderstood requirement deep into Construction (as can happen in Waterfall), UP forces at least a first pass at both requirements and risk identification right at the start, with Elaboration then dedicated specifically to hardening the architecture against those known risks before full-scale Construction begins.

Q6. Why is the Unified Process sometimes described as combining "the strengths of traditional models with agile principles"? What specifically makes it iterative rather than purely prescriptive like Waterfall?

Answer: UP is iterative because it doesn't try to complete the entire system in one linear pass through Inception → Elaboration → Construction → Transition → Production — instead, Construction in particular is understood to happen across multiple increments, each producing a growing, working piece of the system that can be evaluated and adjusted, much like Agile's incremental delivery. It retains traditional-model strengths by still requiring a strong architectural baseline (established explicitly during Elaboration, before large-scale coding starts) and structured phases with defined goals and risk checkpoints, which pure agile methods often leave much lighter or implicit. The combination is the appeal: architecture and risk get the up-front attention that heavyweight, prescriptive models are good at, while the increment-by-increment construction and evaluation gets the adaptability and fast feedback that agile methods are good at.

Q7. Scenario: A team building a small internal tool with a two-week deadline and a single, experienced stakeholder who has given very clear, unlikely-to-change requirements is deciding between Waterfall and Spiral. Which is the better fit, and why would the other model actively hurt them here?

Answer: Waterfall is the better fit here — the requirements are clear, stable, and coming from a single stakeholder, which is exactly the scenario Waterfall's advantages (simplicity, clear milestones, ease of understanding) are designed for, and the small scope means the disadvantages (difficulty accommodating change, late delivery) barely apply since there's little time for requirements to drift and not much to deliver late. The Spiral Model would actively hurt them because its core value comes from repeated, structured risk analysis across multiple cycles — machinery that takes real time and, per its own stated disadvantages, requires experienced developers specifically to execute the risk-management portion well. For a two-week, low-risk, well-understood project, that overhead produces no corresponding benefit; the team would spend meaningful fractions of their short timeline running spiral cycles to manage risks that, in this scenario, barely exist.